Reviewer Pack — Minimal Frobenius Class Representation Size and Communicatio…

Entry fa6e8affd238 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 04:43:13 UTC

Minimal Frobenius Class Representation Size and Communication Complexity Rank Variance

Entry ID: `fa6e8affd238` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 04:43:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Frobenius Classes) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For every CNF φ with n variables, the size of the minimal set of primes generating the Frobenius class of φ is linearly correlated with the variance in communication complexity rank among all possible matrix representations of φ's truth table, such that |P(φ)| = Θ(C(n) * Var(R(φ)))

Rationale (proposer's reasoning):

> Frobenius classes encode arithmetic properties that could reflect hidden structures in boolean functions. If the size of a Frobenius class is related to communication complexity rank variance, it might reveal a connection between arithmetic structure and complexity measures, potentially leading to new insights into circuit complexity lower bounds.

Taxonomy category: `algebraic_number_theory_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fc89147519dabd68`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the computed correlations between |P(φ)| and C(n) * Var(R(φ)) for all CNFs φ with n ≤ 40 are within ±3 standard deviations from the mean, where 'C(n)' is a linear function of n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius class" AND "communication complexity rank variance" - "algebraic number theory" AND "matrix representation of CNF" - "minimal Frobenius class size" AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def truth_table_from_cnf(cnf):
    n = max(abs(lit) for lit in cnf[0])  # Ensure cnf is not empty
    truth_table = [[False] * (2 ** n) for _ in range(len(cnf))]
    for i, clause in enumerate(cnf):
        for assignment in range(1 << n):
            if all(abs(lit) <= n and ((assignment >> abs(lit) - 1) & 1) == (lit > 0) for lit in cl
                truth_table[i][assignment] = True
    return truth_table

def frobenius_class(truth_table):
    primes = []
    for i in range(len(truth_table)):
        if all(row[i] == truth_table[0][i] for row in truth_table):
            primes.append(i + 1)
    return set(primes)

def communication_complexity_rank_variance(truth_table):
    n = len(truth_table[0])
    ranks = [sum(row[i] for row in truth_table) for i in range(n)]
    mean_rank = sum(ranks) / n
    variance = sum((rank - mean_rank) ** 2 for rank in ranks) / n
    return variance

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF with up to 40 variables and clauses
    n = random.randint(5, 40)
    cnf = []
    for _ in range(random.randint(1, 2 * n)):
```

```
        clause = [random.choice([-i, i]) for i in range(1, n + 1)]
        if len(set(clause)) > 1:  # Ensure no duplicate literals
            cnf.append(clause)

    truth_table = truth_table_from_cnf(cnf)
    frobenius_set_size = len(frobenius_class(truth_table))
    rank_variance = communication_complexity_rank_variance(truth_table)

    return {
        "metric_name": "Frobenius Class Size and Rank Variance",
        "metric_value": frobenius_set_size * rank_variance,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,  # Mapping undefined for this conjecture
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))  # Default to first 30 primes if

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd72e43f.py", line 71, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd72e43f.py", line 52, in run_trial
    truth_table = truth_table_from_cnf(cnf)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cd72e43f.py", line 20, in truth_table_f
```

```
    truth_table = [[False] * (2 ** n) for _ in range(len(cnf))]
                  ~~~~~~~~^~~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a MemoryError before producing data, which means it did not complete the computation required to evaluate the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12425
2	propose	ollama_remote	glm4:latest	0	0	17584
3	preregistration	ollama_remote	glm4:latest	0	0	13990
4	novelty	ollama_remote	glm4:latest	0	0	8235
5	novelty	ollama_remote	glm4:latest	0	0	8911
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21328
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14971
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10720
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9676
10	judge	ollama_remote	glm4:latest	0	0	27784

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 145625 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/fa6e8affd238.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa6e8affd238.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fa6e8affd238.tar.gz` (if generated)